Taller propuesto de aprendizaje profundo con KMNIST
Informe de ejecución

Sergio Santiago Duarte Rojas, código 20261595002
Inteligencia Computacional Aplicada, docente Cesar Andrey Perdomo Charry
Universidad Distrital Francisco José de Caldas, 10 de octubre de 2026

1. Ejecución

Se completaron los cinco bloques `TODO` del cuaderno propuesto en Python y se ejecutó completo con KMNIST, caracteres manuscritos japoneses *kuzushiji* de diez clases, 5 épocas, PyTorch 2.12 en GPU y semilla 42. KMNIST se usó real, sin el conjunto de respaldo, con 55 000 imágenes de entrenamiento, 5 000 de validación y 10 000 de prueba. Las diez verificaciones automáticas dieron `[OK]`. La celda opcional que exporta imágenes para MATLAB no se ejecutó porque el trabajo se hizo en Python.

Se agregaron dos celdas. Una muestra ejemplos del par más confundido junto a ejemplos bien clasificados de cada clase, para responder el paso 4. La otra repite con tres semillas la comparación con y sin aumento de datos, como recomienda el aviso del propio cuaderno.

2. Resultados por paso

TODO 1, balance de clases. Las diez clases tienen entre 5 450 y 5 533 imágenes de entrenamiento, así que el conjunto está balanceado y la exactitud global no esconde una clase minoritaria.

TODO 2, arquitectura. La red tiene dos bloques convolucionales y 50 378 parámetros, una cuarta parte del límite de 200 000 (Código 1). Cada componente tiene una razón. La convolución 3×3 detecta trazos locales con los mismos pesos en toda la imagen, y el relleno de 1 píxel conserva el tamaño para que el *pooling* decida la reducción. La normalización por lotes reescala las activaciones de cada canal en cada lote, lo que estabiliza el entrenamiento con la tasa de 10^{-3} desde la primera época. La ReLU aporta la no linealidad. El *max pooling* 2×2 reduce la imagen de 28 a 14 y luego a 7 píxeles por lado y da tolerancia a desplazamientos pequeños. El *dropout* de 0.3 va antes de la capa lineal porque esa capa concentra el 62 % de los parámetros (31 370), y es donde más riesgo hay de memorizar.

Código 1: Arquitectura del TODO 2

```
self.conv1 = nn.Conv2d(1, 32, kernel_size=3, padding=1)     # 28 -> 28
self.bn1 = nn.BatchNorm2d(32)
self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)    # 14 -> 14
self.bn2 = nn.BatchNorm2d(64)
self.drop = nn.Dropout(p_drop)
self.fc = nn.Linear(64 * 7 * 7, 10)

x = F.max_pool2d(F.relu(self.bn1(self.conv1(x))), 2)        # 28 -> 14
x = F.max_pool2d(F.relu(self.bn2(self.conv2(x))), 2)        # 14 -> 7
return self.fc(self.drop(x.flatten(1)))
```

TODO 3, entrenamiento. El bucle sigue el orden pedido, borrar gradientes, propagar, calcular la entropía cruzada, retropropagar y actualizar con Adam. La pérdida de entrenamiento baja de 0.306 a 0.081 y la exactitud de validación llega a 97.56 % (Figura 1). Las curvas de entrenamiento y validación terminan juntas (97.52 y 97.56 %), sin señal de sobreajuste en 5 épocas.

TODO 4, prueba y errores. La exactitud de prueba es 93.16 %, cuatro puntos por debajo de la de validación. Como la validación es una muestra al azar de las imágenes de entrenamiento y las dos

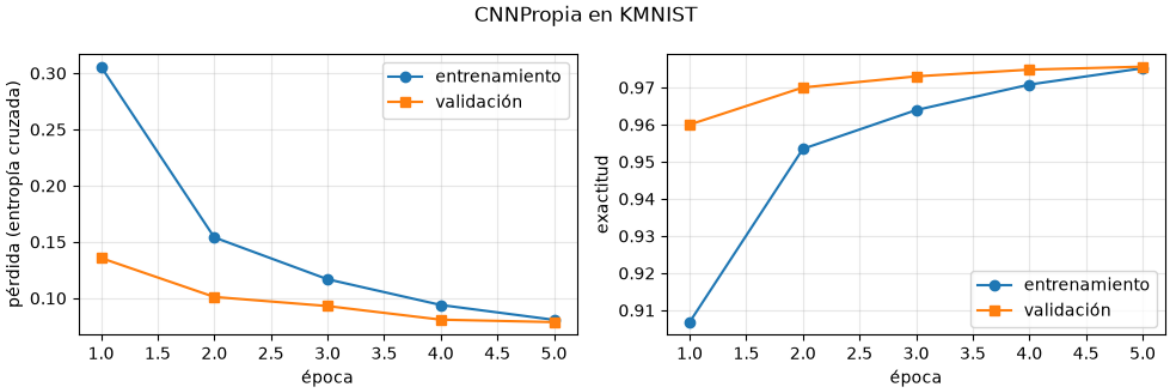


Figura 1: Pérdida y exactitud por época de la red propia en KMNIST

curvas coinciden, la brecha no es sobreajuste, sino que sugiere que las imágenes de prueba difieren de las de entrenamiento, algo que una validación tomada del mismo conjunto no puede detectar.

El par más confundido es *su* tomado por *tsu*, 55 de los 1 000 *su* de prueba (Figura 2). La Figura 3 explica por qué. La clase *su* reúne varias formas antiguas del mismo carácter muy distintas entre sí, y las que la red falla tienen trazos verticales paralelos con un lazo, muy parecidos a la variante de *tsu* hecha de líneas verticales. La matriz muestra además que la columna *na* recibe errores de casi todas las clases, de modo que la red usa *na* como respuesta por defecto ante formas que no reconoce bien.

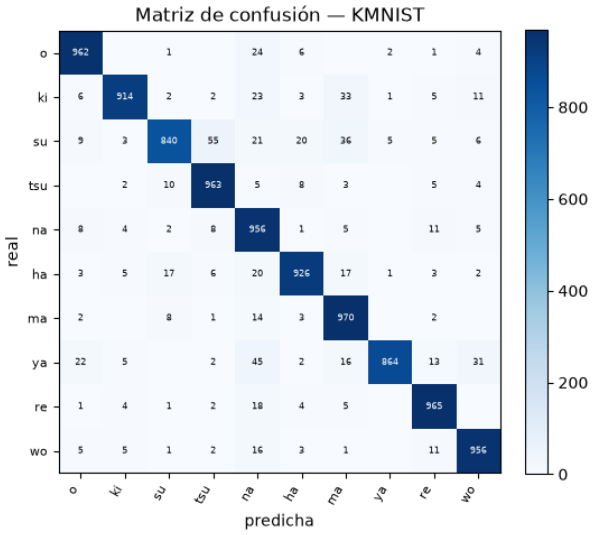


Figura 2: Matriz de confusión en prueba

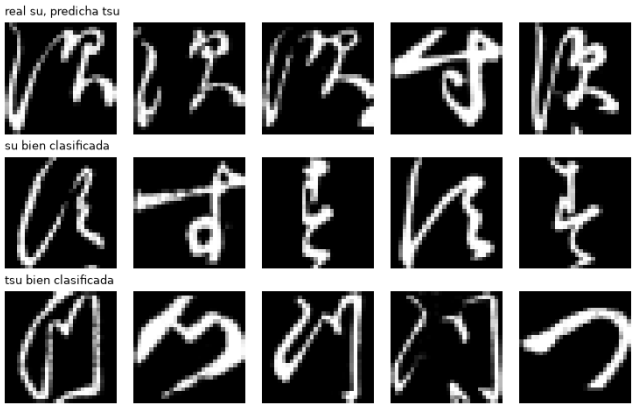


Figura 3: Errores *su* tomado por *tsu* (arriba) y ejemplos bien clasificados de cada clase

TODO 5, aumento de datos. Cada lote se desplaza al azar hasta 2 píxeles en cada eje con `torch.roll` (Código 2). La Tabla 1 resume tres semillas con 5 épocas cada una.

Código 2: Aumento por desplazamiento del TODO 5

```
dy, dx = (int(v) for v in torch.randint(-max_px, max_px + 1, (2,)))
return torch.roll(xb, shifts=(dy, dx), dims=(2, 3))
```

El aumento no mejoró. La validación queda igual y la prueba baja 0.2 puntos, del orden de una desviación entre semillas, así que tampoco empeora de forma clara. Lo que sí cambia es la brecha. Sin

Tabla 1: Exactitud en porcentaje, media y desviación de tres semillas con 5 épocas

	Entrenamiento	**Validación**	**Prueba**
Sin aumento	97.52 ± 0.02	97.75 ± 0.07	93.19 ± 0.24
Con aumento	95.20 ± 0.18	97.75 ± 0.20	92.97 ± 0.20

aumento entrenamiento y validación casi coinciden, y con aumento el entrenamiento cae 2.3 puntos porque la red ve imágenes desplazadas más difíciles, de modo que la validación queda 2.6 puntos por encima. Es el efecto de un regularizador, pero en 5 épocas la red no estaba sobreajustando, así que no había brecha que cerrar. Hay además dos razones posibles para que no ayude en la prueba. El *pooling* ya tolera desplazamientos pequeños, y los errores vienen de variantes de forma como *su* y *tsu*, no de la posición del carácter. Por otro lado, `torch.roll` hace que lo que sale por un borde entre por el opuesto, lo que puede pegar trozos de trazo en el lado contrario.

3. Conclusión

La red propia de dos bloques alcanza 97.6 % de validación y 93.2 % de prueba en KMNIST con 50 378 parámetros. La diferencia entre validación y prueba no se explica por sobreajuste y apunta a que el conjunto de prueba es más difícil que una muestra del de entrenamiento. El error dominante, *su* tomado por *tsu*, nace de clases con varias formas de escritura, y el desplazamiento aleatorio de ± 2 píxeles no lo corrige.